Section 5: Structural Design Patterns

This section is part of the 5-hour LLD course by Mukul Raina

- YouTube Course: https://www.youtube.com/watch?v=VSABVEBDhf0
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

SLIDE 54: Section Intro: Structural Design Patterns

SLIDE 55: Adapter: Integrating Incompatible Interfaces

SLIDE 56: Worked Example: Wrapping a Third-Party Payment API

SLIDE 56B: Adapter in Legacy System Integration

SLIDE 57: Decorator: Adding Behavior Dynamically Without Subclassing

SLIDE 58: Why Decorator Beats Inheritance for Combining Behaviors

SLIDE 58B: Worked Example: Logging, Encryption, and Compression on a Data Stream

SLIDE 58C: Interview Practice: Adapter and Decorator

SLIDE 59: Facade: Simplifying Complex Subsystems

SLIDE 60: Worked Example: OrderFacade Coordinating Services

SLIDE 60B: When Simplification Helps vs When It Hides Complexity

SLIDE 61: Proxy: Controlling Access, Lazy Loading, and Caching

SLIDE 62: Worked Example: Caching Proxy for Database Queries

SLIDE 62B: Virtual Proxy for Lazy-Loading Large Objects

SLIDE 62C: Interview Practice: Facade and Proxy

SLIDE 63: Structural Pattern Decision Framework

SLIDE 63B: Note It Down: Structural Pattern Signals

SLIDE 64: Structural Patterns Interview Practice and Common Mistakes

SLIDE 54: Section Intro: Structural Design Patterns

Section 5: Structural Design Patterns

Structural patterns address how classes and objects are composed to form larger structures. While creational patterns (Section 4) solved how objects are created, structural patterns solve how objects are connected and how their interfaces relate to each other. These patterns let you fit together classes that were not originally designed to work with each other, add behavior to objects without modifying their source code, and simplify complex subsystems behind clean facades.

What We Will Cover

1. Adapter: integrating incompatible interfaces without modifying either
2. Decorator: adding behavior dynamically without subclassing
3. Facade: simplifying complex subsystems behind a clean interface
4. Proxy: controlling access, lazy loading, and caching

Where Structural Patterns Appear in LLD Interviews

Interview Signal	Structural Pattern	Why
“Integrate with a third-party API that has a different interface”	Adapter	Bridge incompatible interfaces
“Add logging/caching/validation without modifying the core class”	Decorator	Layer behavior dynamically
“This subsystem has 6 services, simplify the API”	Facade	One entry point for complex coordination
“Control access to this expensive resource”	Proxy	Lazy loading, caching, access control
“We might swap this external dependency later”	Adapter	Isolate your code from external changes

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

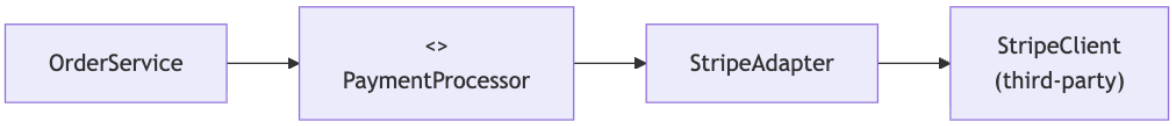
SLIDE 55: Adapter: Integrating Incompatible Interfaces

The Adapter pattern allows two classes with incompatible interfaces to work together by wrapping one class in a translation layer. The adapter converts the interface of the wrapped class into the interface that the calling code expects.

When You Need an Adapter

You need an Adapter when your code depends on an interface, but the concrete implementation you want to use has a different method signature, different parameter types, or a different return type. This happens most commonly when integrating third-party libraries, legacy systems, or external APIs whose interfaces you cannot modify.

The Structure



The `OrderService` depends on `PaymentProcessor` (your interface). The `StripeClient` is a third-party class you cannot modify. The `StripeAdapter` sits between them, translating calls from your interface to the third-party interface.

How Adapter Connects to Previous Sections

Adapter is a direct application of DIP from Section 3. Your business logic depends on your own abstraction (`PaymentProcessor`), and the Adapter handles the translation to a concrete external dependency (`StripeClient`). If Stripe changes their SDK, you update the Adapter. Your business logic is untouched.

Interview Relevance: Adapter appears in any LLD question involving third-party integrations. Interviewers evaluate whether you isolate your code from external dependencies using your own interface, or couple directly to the third-party API.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 56: Worked Example: Wrapping a Third-Party Payment API

Your system defines a `PaymentProcessor` interface. Stripe's SDK has a completely different method structure. The Adapter bridges the gap.

Your Interface

```
public interface PaymentProcessor {
    PaymentResult charge(String customerId, Money amount);
    PaymentResult refund(String transactionId, Money amount);
}
```

Stripe's SDK (Third-Party, Cannot Modify)

```
public class StripeClient {
    public StripeCharge createCharge(long amountInCents, Stri
ng currency,
                                     String customerToken) {
... }
    public StripeRefund createRefund(String chargeId, long am
ountInCents) { ... }
}
```

Stripe uses cents (long) instead of your `Money` object. Stripe calls it `customerToken` instead of `customerId`. Stripe returns `StripeCharge` instead of your `PaymentResult`. Every parameter and return type is different.

The Adapter

```
public class StripePaymentAdapter implements PaymentProcessor
{
    private final StripeClient stripeClient;
    private final CustomerTokenResolver tokenResolver;
```

```
    public StripePaymentAdapter(StripeClient stripeClient,
                                CustomerTokenResolver tokenR
esolver) {
        this.stripeClient = stripeClient;
        this.tokenResolver = tokenResolver;
    }

    @Override
    public PaymentResult charge(String customerId, Money amou
nt) {
        String customerToken = tokenResolver.resolve(customer
Id);
        StripeCharge charge = stripeClient.createCharge(
            amount.toCents(), amount.getCurrencyCode(), custo
merToken
        );
        return mapToPaymentResult(charge);
    }

    @Override
    public PaymentResult refund(String transactionId, Money a
mount) {
        StripeRefund refund = stripeClient.createRefund(trans
actionId, amount.toCents());
        return new PaymentResult(refund.getId(), mapStatus(re
fund.getStatus()));
    }

    private PaymentResult mapToPaymentResult(StripeCharge cha
rge) {
        return new PaymentResult(charge.getId(), mapStatus(ch
arge.getStatus()));
    }

    private PaymentStatus mapStatus(String stripeStatus) {
```

```
        return switch (stripeStatus) {
            case "succeeded" -> PaymentStatus.COMPLETED;
            case "pending" -> PaymentStatus.PENDING;
            default -> PaymentStatus.FAILED;
        };
    }
}
```

The Adapter handles three types of translation: data format conversion (`Money` to cents), identifier mapping (customer ID to Stripe token), and return type mapping (`StripeCharge` to `PaymentResult`). The `OrderService` sees only `PaymentProcessor` and is completely unaware that Stripe exists.

Interview Relevance: Walking through the specific translations (data format, identifiers, return types) shows the interviewer that you understand what the Adapter actually does, not just its structural diagram.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 56B: Adapter in Legacy System Integration

Adapter is equally valuable when integrating with legacy internal systems. Consider a modern order management system that needs to read customer data from a legacy SOAP-based CRM that the company cannot yet replace.

The Legacy System

```
public class LegacySoapCrmClient {
    public SoapResponse getCustomerBySoapId(String soapCustom
erId) { ... }
    public SoapResponse updateCustomerAddress(String soapCust
omerId,
                                              SoapAddress ad
dress) { ... }
}
```

Your Modern Interface

```
public interface CustomerRepository {
    Optional<Customer> findById(String customerId);
    void updateAddress(String customerId, Address address);
}
```

The Adapter

```
public class LegacyCrmAdapter implements CustomerRepository {
    private final LegacySoapCrmClient crmClient;
    private final CustomerIdMapper idMapper;

    public LegacyCrmAdapter(LegacySoapCrmClient crmClient,
                            CustomerIdMapper idMapper) {
        this.crmClient = crmClient;
        this.idMapper = idMapper;
    }

    @Override
    public Optional<Customer> findById(String customerId) {
        String soapId = idMapper.toSoapId(customerId);
        SoapResponse response = crmClient.getCustomerBySoapId
(soapId);
        if (response.isNotFound()) {
            return Optional.empty();
        }
        return Optional.of(mapToCustomer(response));
    }

    @Override
    public void updateAddress(String customerId, Address addr
ess) {
        String soapId = idMapper.toSoapId(customerId);
        SoapAddress soapAddress = new SoapAddress(
            address.getStreet(), address.getCity(),
```

```
                address.getState(), address.getPostalCode()
            );
            crmClient.updateCustomerAddress(soapId, soapAddress);
        }
    }
```

When the company eventually replaces the legacy CRM with a modern REST API, only the Adapter changes. Every class that depends on `CustomerRepository` is untouched. This is the real-world value of Adapter: it creates a seam between your codebase and systems you cannot control.

Interview Tip: In an LLD interview, whenever you integrate with an external or legacy system, wrap it in an Adapter behind your own interface. State explicitly: "I am wrapping the external dependency so that if it changes, only the adapter needs to be updated." This demonstrates both Adapter pattern knowledge and DIP compliance.

Interview Relevance: Legacy integration is a common LLD interview theme. Interviewers test whether you couple your design directly to the legacy interface or isolate it behind an adapter.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

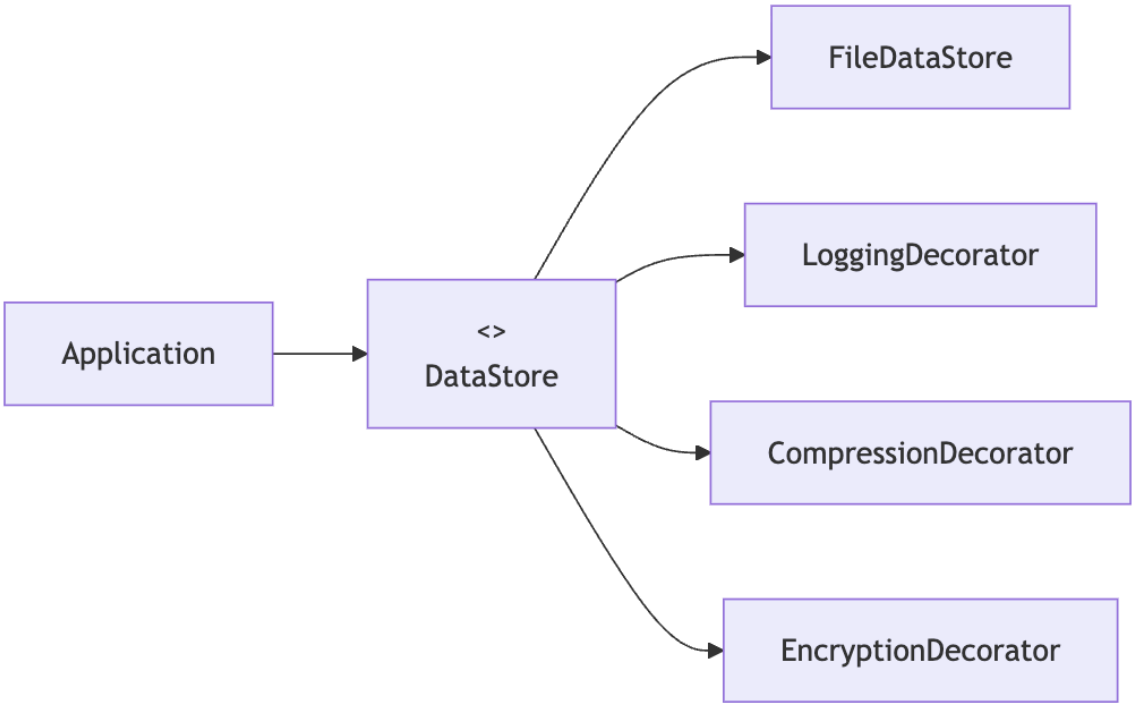
SLIDE 57: Decorator: Adding Behavior Dynamically Without Subclassing

The Decorator pattern lets you attach additional behavior to an object at runtime by wrapping it in another object that has the same interface. Each decorator adds one layer of behavior and delegates the core work to the wrapped object.

The Core Idea

Consider a data storage service. The basic version reads and writes data. You need to add optional capabilities: logging every operation, compressing data before storage, and encrypting sensitive data. With inheritance, you would need classes like `LoggingEncryptingCompressingDataStore`, and every combination would be a separate class. With Decorator, you stack the behaviors you need at runtime.

The Structure



Every decorator implements the same `DataStore` interface and holds a reference to another `DataStore`. This means decorators can wrap the base implementation or wrap other decorators, creating a chain.

The Key Difference from Inheritance

With inheritance, the combination of behaviors is fixed at compile time. With Decorator, you choose which behaviors to combine at runtime. You can have a `LoggingDecorator` wrapping a `FileDataStore` in one context and a `LoggingDecorator` wrapping a `CompressionDecorator` wrapping an `EncryptionDecorator` wrapping a `FileDataStore` in another context, all using the same classes.

Interview Relevance: Decorator is tested when an interviewer asks “how would you add cross-cutting concerns like logging or caching without modifying the core class?” The answer is Decorator, not inheritance.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 58: Why Decorator Beats Inheritance for Combining Behaviors

The Inheritance Explosion Problem (from Section 2, Slide 21)

If you have 1 base class and 3 optional behaviors (logging, compression, encryption), inheritance requires a class for every combination:

Class	Behaviors
`FileDataStore`	None
`LoggingFileDataStore`	Logging
`CompressingFileDataStore`	Compression
`EncryptingFileDataStore`	Encryption
`LoggingCompressingFileDataStore`	Logging + Compression
`LoggingEncryptingFileDataStore`	Logging + Encryption
`CompressingEncryptingFileDataStore`	Compression + Encryption
`LoggingCompressingEncryptingFileDataStore`	All three

That is 7 subclasses for 3 optional behaviors. Adding a fourth behavior (caching) doubles it to 15 subclasses. The count grows as 2^n where n is the number of optional behaviors.

Decorator: Linear Growth

With Decorator, you write one class per behavior (3 decorators for 3 behaviors, 4 for 4 behaviors). The combinations are assembled at runtime:

```
DataStore simpleStore = new FileDataStore("/data");

DataStore loggedStore = new LoggingDecorator(new FileDataStor
e("/data"));

DataStore fullStore = new LoggingDecorator(
```

```
        new CompressionDecorator(
            new EncryptionDecorator(
                new FileDataStore("/data")
            )
        )
    );
```

Three decorators handle all 7 combinations. Adding a fourth behavior (caching) means writing one new class, not 8.

Why This Matters for Design

Dimension	Inheritance	Decorator
Classes needed for n behaviors	2^n - 1	n
Combination flexibility	Fixed at compile time	Assembled at runtime
Adding a new behavior	Doubles the class count	One new class
Removing a behavior	Cannot selectively remove	Omit the decorator from the chain
Testing individual behaviors	Must test in full hierarchy	Test each decorator in isolation

Interview Relevance: If an interviewer asks “how would you add logging, caching, and rate limiting to this service?”, Decorator is the answer. State the 2^n vs n class count trade-off to demonstrate that you understand why Decorator exists, not just how it works.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 58B: Worked Example: Logging, Encryption, and Compression on a Data Stream

The Interface

```
public interface DataStore {
    void write(String key, byte[] data);
```

```
    byte[] read(String key);
}
```

The Base Implementation

```
public class FileDataStore implements DataStore {
    private final String basePath;

    public FileDataStore(String basePath) {
        this.basePath = basePath;
    }

    @Override
    public void write(String key, byte[] data) {
        Path filePath = Path.of(basePath, key);
        Files.write(filePath, data);
    }

    @Override
    public byte[] read(String key) {
        Path filePath = Path.of(basePath, key);
        return Files.readAllBytes(filePath);
    }
}
```

The Decorators

```
public class LoggingDecorator implements DataStore {
    private final DataStore delegate;
    private final Logger logger;

    public LoggingDecorator(DataStore delegate, Logger logge
r) {
        this.delegate = delegate;
        this.logger = logger;
    }
```

```
    @Override
    public void write(String key, byte[] data) {
        logger.info("Writing {} bytes to key: {}", data.lengt
h, key);
        long startTime = System.currentTimeMillis();
        delegate.write(key, data);
        long duration = System.currentTimeMillis() - startTim
e;
        logger.info("Write completed in {} ms", duration);
    }

    @Override
    public byte[] read(String key) {
        logger.info("Reading key: {}", key);
        byte[] data = delegate.read(key);
        logger.info("Read {} bytes from key: {}", data.lengt
h, key);
        return data;
    }
}
```

```
public class EncryptionDecorator implements DataStore {
    private final DataStore delegate;
    private final EncryptionService encryptionService;

    public EncryptionDecorator(DataStore delegate,
                               EncryptionService encryptionS
ervice) {
        this.delegate = delegate;
        this.encryptionService = encryptionService;
    }

    @Override
    public void write(String key, byte[] data) {
```

```
        byte[] encrypted = encryptionService.encrypt(data);
        delegate.write(key, encrypted);
    }

    @Override
    public byte[] read(String key) {
        byte[] encrypted = delegate.read(key);
        return encryptionService.decrypt(encrypted);
    }
}
```

```
public class CompressionDecorator implements DataStore {
    private final DataStore delegate;

    public CompressionDecorator(DataStore delegate) {
        this.delegate = delegate;
    }

    @Override
    public void write(String key, byte[] data) {
        byte[] compressed = GzipCompressor.compress(data);
        delegate.write(key, compressed);
    }

    @Override
    public byte[] read(String key) {
        byte[] compressed = delegate.read(key);
        return GzipCompressor.decompress(compressed);
    }
}
```

Assembling the Chain

```
DataStore store = new LoggingDecorator(
    new CompressionDecorator(
```

```
            new EncryptionDecorator(
                new FileDataStore("/data"),
                new AesEncryptionService(secretKey)
            )
        ),
        LoggerFactory.getLogger("DataStore")
    );

    store.write("user-profile-42", profileBytes);
```

The execution order for a write: Logging logs the call, then Compression compresses the data, then Encryption encrypts it, then FileDataStore writes to disk. For a read, the order reverses: FileDataStore reads, Encryption decrypts, Compression decompresses, Logging logs. Each decorator is independently testable, independently removable, and independently reusable.

Interview Relevance: This exact pattern (interface + base + decorators) is the standard Decorator implementation. Showing the full chain assembly in your interview demonstrates that you understand how decorators compose.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 58C: Interview Practice: Adapter and Decorator

Scenario

You are designing a metrics collection system for a platform like Datadog. The system needs to record metrics (counters, timers, gauges) to multiple backends: an in-memory store for testing, a StatsD server for production, and a cloud monitoring API for long-term storage. Additionally, all metrics should optionally support: tagging (adding key-value labels), batching (buffering metrics and flushing periodically), and sampling (recording only a percentage of metrics to reduce load).

The interviewer asks: “How would you structure this?”

Pause the video and identify which patterns apply for 60 seconds before continuing.

Weak Answer

"I would create a `MetricsService` class with methods for each backend and boolean flags for tagging, batching, and sampling."

This combines all backends and all optional behaviors into one class. Adding a new backend or a new optional behavior requires modifying the same class. This violates SRP, OCP, and makes the class untestable.

Strong Answer

"The multiple backends (in-memory, StatsD, cloud API) are an Adapter problem. I would define a `MetricsRecorder` interface and write an Adapter for each backend. Each adapter translates from my interface to the backend's API.

The optional behaviors (tagging, batching, sampling) are a Decorator problem. Each is a cross-cutting concern that wraps a `MetricsRecorder` without modifying it. A `TaggingDecorator` adds labels, a `BatchingDecorator` buffers and flushes, and a `SamplingDecorator` probabilistically drops metrics.

In production, the chain would be: `SamplingDecorator` wrapping `BatchingDecorator` wrapping `TaggingDecorator` wrapping `StatsDAdapter`. In tests: just `InMemoryMetricsRecorder` with no decorators."

The strong answer identifies two distinct problems, applies the correct pattern to each, and explains how they compose together.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 59: Facade: Simplifying Complex Subsystems

The Facade pattern provides a simplified interface to a complex subsystem. Instead of forcing calling code to coordinate multiple services, manage sequencing, and handle error cases across several classes, the Facade wraps the subsystem behind a single class with a straightforward API.

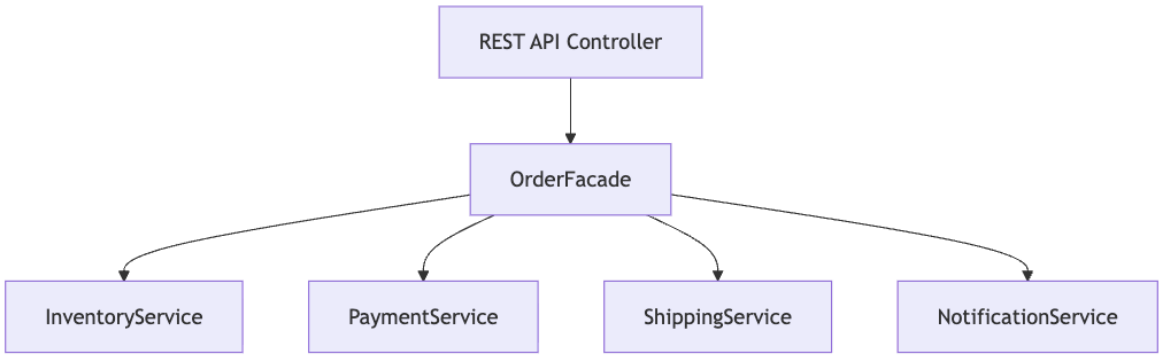
When You Need a Facade

Facade is appropriate when a workflow involves coordinating 3+ services in a specific sequence, and multiple callers need to perform the same coordination. Without a Facade, the coordination logic is duplicated across every caller.

The Problem: Scattered Coordination

Consider placing an order on a platform like Shopify. The workflow requires: checking inventory, validating the payment method, charging the payment, reserving inventory, creating a shipping label, and sending a confirmation email. Without a Facade, every endpoint or service that places an order must coordinate all six steps and handle partial failures (what if payment succeeds but shipping label creation fails?).

The Solution: OrderFacade



The API controller calls one method on `OrderFacade`. The Facade coordinates all subsystems, handles sequencing, and manages error recovery. The controller does not know how many services are involved or in what order they are called.

Interview Relevance: Facade appears whenever an interviewer asks you to design an API for a multi-step workflow. The key signal is "this operation involves coordinating multiple services." Extracting a Facade shows the interviewer that you think about API simplicity, not just internal design.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 60: Worked Example: OrderFacade Coordinating Services

```
public class OrderFacade {
    private final InventoryService inventoryService;
    private final PaymentService paymentService;
    private final ShippingService shippingService;
    private final NotificationService notificationService;

    public OrderFacade(InventoryService inventoryService,
                       PaymentService paymentService,
                       ShippingService shippingService,
                       NotificationService notificationServic
e) {
        this.inventoryService = inventoryService;
        this.paymentService = paymentService;
        this.shippingService = shippingService;
        this.notificationService = notificationService;
    }

    public OrderResult placeOrder(OrderRequest request) {
        InventoryCheck inventoryCheck = inventoryService.chec
kAvailability(request.getItems());
        if (!inventoryCheck.isAvailable()) {
            return OrderResult.outOfStock(inventoryCheck.getU
navailableItems());
        }

        PaymentResult paymentResult = paymentService.charge(
            request.getCustomerId(), request.getTotalAmount()
        );
        if (!paymentResult.isSuccessful()) {
            return OrderResult.paymentFailed(paymentResult.ge
tErrorMessage());
        }

        inventoryService.reserve(request.getItems());
```

```
        ShippingLabel label;
        try {
            label = shippingService.createLabel(
                request.getShippingAddress(), request.getItem
s()
            );
        } catch (ShippingException e) {
            paymentService.refund(paymentResult.getTransactio
nId());
            inventoryService.release(request.getItems());
            return OrderResult.shippingFailed(e.getMessage
());
        }

        Order order = new Order(request, paymentResult.getTra
nsactionId(), label.getTrackingNumber());
        notificationService.sendOrderConfirmation(request.get
CustomerId(), order);

        return OrderResult.success(order.getId(), label.getTr
ackingNumber());
    }
}
```

What the Facade Handles

The Facade manages three concerns that calling code should not deal with:

1. **Sequencing:** Inventory must be checked before payment, payment before shipping, shipping before notification. The order matters and the Facade enforces it.
2. **Error recovery:** If shipping fails after payment succeeds, the Facade refunds the payment and releases the inventory reservation. This compensating transaction logic lives in one place, not in every caller.

3. **Simplification:** The caller sees `placeOrder(request)` and gets back a result. It does not know about inventory checks, payment processing, shipping labels, or error recovery sequences.

Interview Tip: When presenting a Facade in an LLD interview, explicitly point out the error recovery logic. Interviewers are impressed when candidates think about partial failure scenarios ("what happens if step 3 fails after step 2 succeeded?") because this is where production systems actually break.

Interview Relevance: The multi-step workflow with compensating transactions is a common LLD interview pattern. Facade is the structural answer; Saga is the distributed systems answer. Knowing both, and when each applies, demonstrates cross-domain design thinking.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 60B: When Simplification Helps vs When It Hides Complexity

Facade simplifies the interface to a subsystem, but simplification has limits. A Facade that hides too much can prevent callers from doing legitimate work.

Facade Done Right

The `OrderFacade` exposes `placeOrder()` and hides coordination details. The caller does not need to know about inventory checks or shipping label creation. The simplification matches the caller's intent: "I want to place an order."

Facade Done Wrong

```
public class DatabaseFacade {
    public List<Map<String, Object>> getData(String entity) {
        return database.query("SELECT * FROM " + entity);
    }
}
```

This Facade is too simplistic. It hides the ability to filter, sort, paginate, or join data. Any caller that needs a WHERE clause must bypass the Facade entirely, which

defeats its purpose. A useful Facade still exposes enough control for the caller to express its intent.

Decision Framework

Signal	Facade Is Appropriate	Facade Is Inappropriate
Callers always need the same workflow	Yes, standardize it	No, if workflows vary significantly
Coordination sequence is complex	Yes, centralize the sequencing	No, if the sequence is trivial (2 calls)
Error recovery involves compensating actions	Yes, encapsulate the recovery logic	Not needed for idempotent operations
Callers need fine-grained control	Expose Facade + direct access	Do not force everything through the Facade
Subsystem is stable and well-defined	Yes, safe to abstract	Be cautious if subsystem is evolving rapidly

The Rule

A Facade should simplify the common case without preventing the uncommon case. Expose the Facade for the 80% of callers who want the standard workflow. Keep the underlying services accessible for the 20% who need finer control.

Interview Relevance: Interviewers sometimes probe whether your Facade is too rigid. Being able to articulate when a Facade helps versus when it over-simplifies shows nuanced design judgment.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 61: Proxy: Controlling Access, Lazy Loading, and Caching

The Proxy pattern provides a surrogate or placeholder for another object to control access to it. The proxy has the same interface as the real object, so calling code cannot tell the difference. The proxy intercepts calls and adds behavior before or after delegating to the real object.

Three Common Proxy Types

Caching Proxy: Stores results of expensive operations and returns the cached value for repeated calls, avoiding redundant computation or database queries.

Virtual Proxy (Lazy Loading): Delays the creation or loading of an expensive object until it is actually needed. The proxy holds a reference that is initially null and creates the real object on first access.

Protection Proxy (Access Control): Checks whether the caller has permission before delegating to the real object. Separates authorization logic from business logic.

How Proxy Differs from Decorator

Dimension	Proxy	Decorator
Primary purpose	Controls access to the object	Adds behavior to the object
Relationship to wrapped object	Often creates or manages the wrapped object's lifecycle	Always receives the wrapped object from outside
Typical use case	Caching, lazy loading, access control	Logging, compression, encryption
Number of wrappers	Usually one proxy per object	Often multiple decorators stacked
Who knows about the wrapping?	Calling code usually does not know	Calling code assembles the decorator chain

The distinction matters because interviewers may ask you to justify why you chose Proxy over Decorator or vice versa. The answer depends on whether you are controlling access or adding behavior.

Interview Relevance: Proxy appears in LLD interviews involving caching, authorization, or expensive resource management. The interviewer tests whether you can separate the access-control concern from the business logic.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 62: Worked Example: Caching Proxy for Database Queries

Consider a product catalog service for a platform like Amazon. Product details are read far more frequently than they are updated. Without caching, every product page view hits the database. A caching proxy eliminates redundant database queries without modifying the repository class.

The Interface

```
public interface ProductRepository {
    Optional<Product> findById(String productId);
    List<Product> findByCategory(String category);
}
```

The Real Implementation

```
public class PostgresProductRepository implements ProductRepo
sitory {
    private final DataSource dataSource;

    public PostgresProductRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Override
    public Optional<Product> findById(String productId) {
        Connection conn = dataSource.getConnection();
        PreparedStatement stmt = conn.prepareStatement(
            "SELECT id, name, price, category FROM products W
HERE id = ?"
        );
        stmt.setString(1, productId);
        ResultSet rs = stmt.executeQuery();
        if (rs.next()) {
            return Optional.of(mapToProduct(rs));
        }
        return Optional.empty();
    }
```

```
    @Override
    public List<Product> findByCategory(String category) {
        Connection conn = dataSource.getConnection();
        PreparedStatement stmt = conn.prepareStatement(
            "SELECT id, name, price, category FROM products W
HERE category = ?"
        );
        stmt.setString(1, category);
        return mapToProducts(stmt.executeQuery());
    }
}
```

The Caching Proxy

```
public class CachingProductRepository implements ProductRepos
itory {
    private final ProductRepository delegate;
    private final Cache<String, Optional<Product>> productCac
he;
    private final Cache<String, List<Product>> categoryCache;

    public CachingProductRepository(ProductRepository delegat
e,
                                    Duration cacheDuration)
{
        this.delegate = delegate;
        this.productCache = CacheBuilder.newBuilder()
            .expireAfterWrite(cacheDuration)
            .maximumSize(10_000)
            .build();
        this.categoryCache = CacheBuilder.newBuilder()
            .expireAfterWrite(cacheDuration)
            .maximumSize(500)
            .build();
    }
```

```
    @Override
    public Optional<Product> findById(String productId) {
        Optional<Product> cached = productCache.getIfPresent
(productId);
        if (cached != null) {
            return cached;
        }
        Optional<Product> product = delegate.findById(product
Id);
        productCache.put(productId, product);
        return product;
    }

    @Override
    public List<Product> findByCategory(String category) {
        List<Product> cached = categoryCache.getIfPresent(cat
egory);
        if (cached != null) {
            return cached;
        }
        List<Product> products = delegate.findByCategory(cate
gory);
        categoryCache.put(category, products);
        return products;
    }
}
```

Wiring

```
ProductRepository repository = new CachingProductRepository(
    new PostgresProductRepository(dataSource),
    Duration.ofMinutes(5)
);
```

The `CatalogService` depends on `ProductRepository`. It does not know whether it is talking to the database or the cache. The caching logic is completely separate from the database logic, each independently testable and modifiable.

Interview Relevance: Caching Proxy is the most common Proxy variant in LLD interviews. Showing the cache-aside pattern (check cache, miss, query database, populate cache) with proper TTL configuration demonstrates production-level thinking.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 62B: Virtual Proxy for Lazy-Loading Large Objects

A virtual proxy delays the creation of an expensive object until it is first accessed. This is useful when loading the object is costly (large file, remote API call, complex initialization) and the object may not always be needed.

The Problem

Consider a document management system where each document has a large binary attachment (PDF, image). Loading all attachments into memory when listing documents wastes resources because most users only preview titles and metadata.

The Virtual Proxy

```
public interface DocumentAttachment {
    byte[] getContent();
    String getContentType();
    long getFileSize();
}
```

```
public class LazyDocumentAttachment implements DocumentAttach
ment {
    private final String attachmentId;
    private final String contentType;
```

```
    private final long fileSize;
    private final AttachmentStore attachmentStore;
    private byte[] cachedContent;

    public LazyDocumentAttachment(String attachmentId, String
contentType,
                                  long fileSize, AttachmentS
tore attachmentStore) {
        this.attachmentId = attachmentId;
        this.contentType = contentType;
        this.fileSize = fileSize;
        this.attachmentStore = attachmentStore;
    }

    @Override
    public byte[] getContent() {
        if (cachedContent == null) {
            cachedContent = attachmentStore.loadContent(attac
hmentId);
        }
        return cachedContent;
    }

    @Override
    public String getContentType() {
        return contentType;
    }

    @Override
    public long getFileSize() {
        return fileSize;
    }
}
```

Metadata fields (`contentType`, `fileSize`) are available immediately because they are loaded from the database index. The actual binary content is loaded only when

`getContent()` is called for the first time. If the user never opens the attachment, the content is never loaded.

Where Virtual Proxy Appears

System	What Is Lazy-Loaded	Why
Document management	File attachments (10+ MB each)	Listing documents should not load all attachments
Social media feed	High-resolution images	Thumbnails are loaded first, full images on click
ORM frameworks	Related entities (Hibernate lazy loading)	Loading an Order should not load all OrderItems automatically
IDE file browsers	File contents	Showing file names should not read all file contents

Interview Tip: When designing a system that manages large objects (files, images, blobs), mention lazy loading proactively. "The `Document` object holds a `LazyAttachment` proxy that loads the binary content only when the user requests it." This shows the interviewer that you consider memory efficiency in your design, not just correctness.

Interview Relevance: Virtual Proxy appears in any LLD question involving large objects, file management, or media systems. It demonstrates that you think about resource efficiency.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 62C: Interview Practice: Facade and Proxy

Scenario

You are designing a user registration flow for a platform like LinkedIn. The flow involves: validating input fields, checking if the email is already registered, hashing the password, creating the user record, sending a verification email, and logging the registration event to an analytics service. The analytics service is expensive to initialize (requires loading ML models for user segmentation) and is

not needed for every registration path (some registrations skip analytics in certain regions).

The interviewer asks: “How would you structure this?”

Pause the video and identify which patterns apply for 60 seconds before continuing.

Weak Answer

“I would put all the steps in a `RegistrationService` class and add an `if` check for analytics.”

This places six concerns in one class (SRP violation) and makes the analytics lazy-loading logic part of the registration flow.

Strong Answer

“The multi-step registration workflow is a Facade problem. I would create a `RegistrationFacade` that coordinates `InputValidator`, `UserRepository`, `PasswordService`, `EmailService`, and `AnalyticsService` in sequence, with error recovery if the email send fails after user creation.

The analytics service’s expensive initialization is a Virtual Proxy problem. I would create an `AnalyticsServiceProxy` that implements the `AnalyticsService` interface but delays the ML model loading until the first analytics call. In regions where analytics is skipped, the models are never loaded.

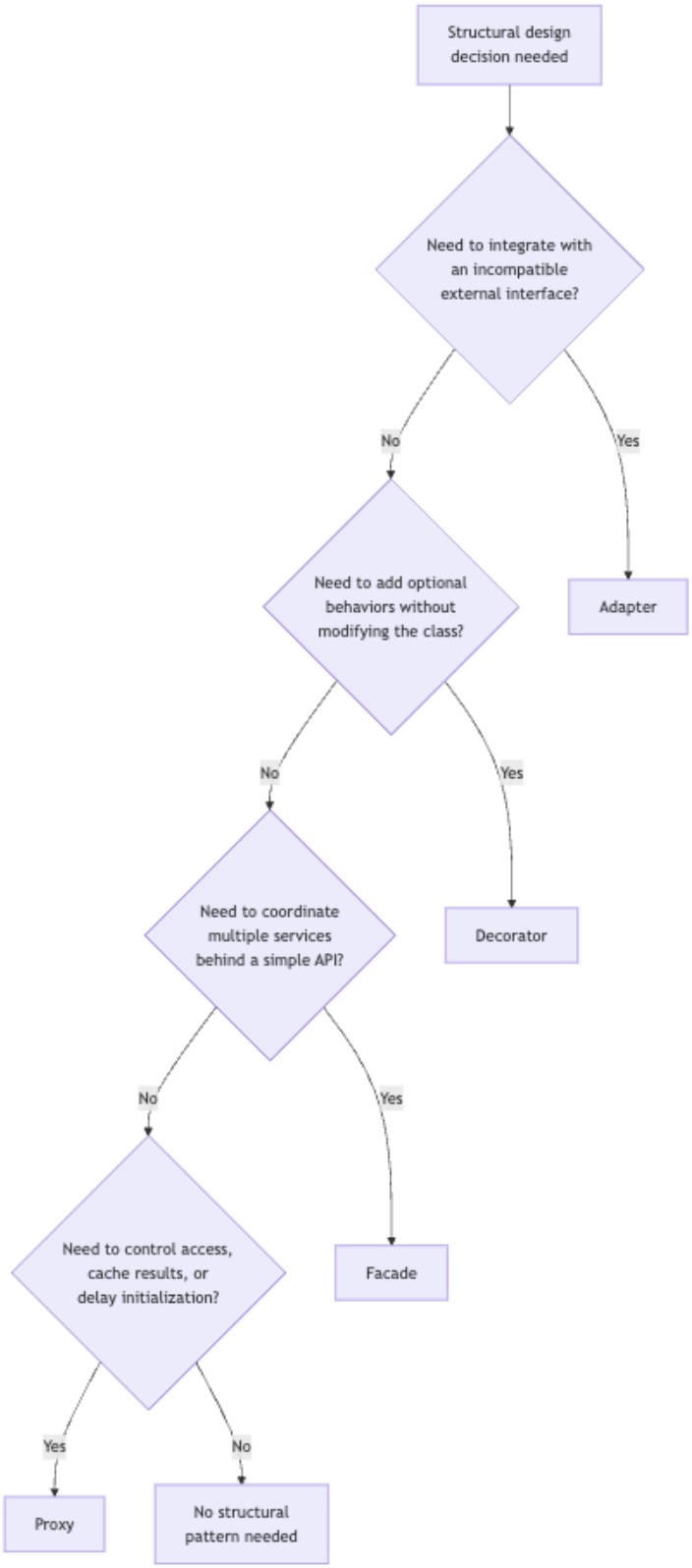
The Facade handles coordination and sequencing. The Proxy handles the lazy-loading concern. Each pattern solves a different problem in the same system.”

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 63: Structural Pattern Decision Framework

Use this decision tree when choosing a structural pattern in an LLD interview.


Structural design decision needed
Need to integrate with an incompatible external interface?
No
Yes
Adapter
Need to add optional behaviors without modifying the class?
No
Yes
Decorator
Need to coordinate multiple services behind a simple API?
No
Yes
Facade
Need to control access, cache results, or delay initialization?
Yes
No
Proxy
No structural pattern needed

Pattern Comparison at a Glance

Pattern	Purpose	Same Interface?	Creates Wrapped Object?	Key Signal
Adapter	Translate between interfaces	No (that is the point)	No (wraps existing)	"This API does not match our interface"
Decorator	Add behavior dynamically	Yes	No (receives from outside)	"Add X without modifying the class"
Facade	Simplify subsystem access	No (provides new, simpler API)	No (coordinates existing)	"Coordinate these 4 services in one call"
Proxy	Control access to object	Yes	Sometimes (lazy init)	"Cache this," "lazy load," "check permissions"

Interview Relevance: In an LLD interview, you may use multiple structural patterns in a single design. Adapter for external integrations, Decorator for cross-cutting concerns, Facade for workflow coordination, and Proxy for caching. Knowing when to reach for each one, and being able to explain why, is the differentiator.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 63B: Note It Down: Structural Pattern Signals

These are the signals that tell you which structural pattern to reach for during an LLD interview. Memorize the mapping from signal to pattern.

Signal-to-Pattern Quick Reference

Signal in the Requirements	Pattern	One-Line Justification
"Integrate with third-party X"	Adapter	"I wrap external APIs behind my own interface"

Signal in the Requirements	Pattern	One-Line Justification
“We might switch from provider A to provider B”	Adapter	“Only the adapter changes, all callers are untouched”
“Add logging/caching/metrics to existing services”	Decorator	“Each concern is a separate decorator, stackable at runtime”
“This operation involves coordinating 4+ services”	Facade	“One entry point, centralized sequencing and error recovery”
“This resource is expensive to load”	Virtual Proxy	“Lazy-load on first access, skip if never needed”
“Cache results of expensive queries”	Caching Proxy	“Check cache first, query on miss, store result”
“Check permissions before accessing this resource”	Protection Proxy	“Authorization is separate from business logic”
“Legacy system has an outdated API”	Adapter	“Adapter translates legacy calls to modern interface”

Three Questions to Ask Yourself

1. **Is the interface incompatible?** If yes, Adapter.
2. **Do I need to add optional, stackable behavior?** If yes, Decorator.
3. **Am I coordinating multiple services or controlling access to one?** Coordinating: Facade. Controlling access: Proxy.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 64: Structural Patterns Interview Practice and Common Mistakes

Practice Scenario

You are designing a reporting API for a platform like Salesforce. The API must: pull data from three internal services (CRM, billing, usage analytics), aggregate and format the data, optionally export to PDF or CSV, and cache results for 15 minutes since reports are expensive to generate. Additionally, the CRM service uses a legacy SOAP interface while your system uses REST-style interfaces.

The interviewer asks: “Walk me through your structural design.”

Pause the video and design the structure for 90 seconds before continuing.

Key Design Decisions

The legacy CRM SOAP interface is an Adapter problem. The coordination of three services is a Facade problem. The caching of expensive reports is a Caching Proxy problem. The optional export formats (PDF, CSV) are a Strategy problem (Section 6) with a Factory to create the right exporter (Section 4).

A strong answer identifies all four patterns and explains where each applies without over-engineering any of them.

Common Structural Pattern Mistakes

Mistake	Why It Is Wrong	Stronger Approach
Coupling directly to a third-party SDK	If the SDK changes, your entire codebase is affected	Wrap in an Adapter behind your own interface
Using inheritance to add logging to 5 classes	Creates 5 subclasses, each duplicating logging logic	One LoggingDecorator reusable across all 5
Facade that hides everything with no direct access	Callers needing fine-grained control must bypass the Facade	Expose Facade for common cases, keep services accessible
Caching inside the repository class	Mixes caching concern with data access	Caching Proxy wrapping the repository
Using Decorator when Proxy is appropriate	Adding “access control” is not the same as “adding behavior”	Use Proxy for access control, Decorator for behavior enhancement
Using Proxy when you just need DI	Wrapping an object just to inject it is unnecessary complexity	Constructor injection is sufficient

Interview Relevance: Structural pattern mistakes often come from choosing the wrong pattern for the problem. The mistake table above maps the most common misapplications to their corrections.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 65: Section Summary and Cheat Sheet

Concepts Covered

Pattern	One-Line Summary
Adapter	Translates an incompatible interface to match your expected interface
Decorator	Adds stackable behavior at runtime without modifying the wrapped class
Facade	Simplifies a complex subsystem behind a single coordinating class
Caching Proxy	Caches results of expensive calls, transparent to the caller
Virtual Proxy	Delays creation of expensive objects until first access
Protection Proxy	Checks permissions before delegating to the real object

Common Interview Mistakes

Mistake	Stronger Approach
“I coupled directly to Stripe’s SDK throughout my codebase”	“I defined my own PaymentProcessor interface and wrote a StripeAdapter”
“I added logging by subclassing every service”	“I wrote one LoggingDecorator that wraps any service implementing the interface”
“I put the caching logic inside the database repository”	“I wrote a CachingProxy that wraps the repository, separating caching from data access”
“I put all 6 steps of the workflow in the controller”	“I created an OrderFacade that coordinates all services and handles error recovery”

Key Points to Remember

Pattern	Recognition Signal	Key Benefit
Adapter	“This external API does not match our interface”	Isolates your code from external changes
Decorator	“Add X behavior without modifying the core class”	Stackable, linear growth instead of 2^n classes
Facade	“Coordinate these N services in the right sequence”	Centralized sequencing, error recovery, simple API
Proxy	“Cache this / lazy-load this / check permissions for this”	Transparent to callers, separates cross-cutting concern

How This Connects to Other Sections

- **Section 2 (OOP):** Decorator relies on polymorphism and composition over inheritance. Adapter applies abstraction to isolate external dependencies.
- **Section 3 (SOLID):** Adapter enables DIP by wrapping external dependencies. Decorator enables OCP by adding behavior without modification. Facade applies SRP to subsystem coordination.
- **Section 4 (Creational):** Adapters are often created by Factory Methods. Decorator chains can be assembled using a Builder-like fluent API.
- **Section 6 (Behavioral):** Strategy objects often sit behind an Adapter when they wrap external algorithms. Chain of Responsibility is structurally similar to Decorator (pipeline of handlers).
- **Section 7 (Pattern Selection):** The structural pattern signals from this section feed directly into the signal-to-pattern mapping table for rapid interview pattern recognition.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn